2 Algorithmique

Prérequis à réactiver

Effectuer des calculs dans le bon ordre; lire un tableau à double entrée; suivre une consigne composée de plusieurs étapes.

Automatismes

Partir de $x = 3$ et exécuter successivement : multiplier par 2, ajouter 5, remplacer le résultat par son carré. Recommencer avec $x = -1$.

Activité d'entrée – Faire exécuter sans ambiguïté

Un élève décrit oralement la construction d'une figure très simple sans la montrer; un autre l'exécute. Comparer la figure obtenue à la figure cible et repérer les instructions ambiguës. Réécrire la procédure avec des données d'entrée, des instructions ordonnées et un résultat contrôlable.

L'origine du mot « algorithme »

*Le mot **algorithme** vient du nom du mathématicien perse Muhammad ibn Mûsâ **al-Khwârizmî** (780–850), dont le nom latinement translittéré donna Algoritmi. Ce même savant donna son nom à une autre discipline : l'**algèbre**, de al-jabr, titre de son traité fondateur. Un homme, deux sciences.*

Objectifs du chapitre

- Comprendre la notion d'algorithme et ses propriétés.
- Lire, exécuter et tracer un algorithme en pseudo-code.

- Tracer et lire un organigramme.
- Écrire un algorithme simple pour un problème mathématique.

2.1 Qu'est-ce qu'un algorithme ?

Définition 2.1 – *Algorithme*

Un **algorithme** est une suite finie et non ambiguë d'instructions permettant de résoudre un problème. Il doit être : **finie** (il termine), **valide** (il donne le bon résultat), et **déterministe** (même entrée $\Rightarrow$ même sortie).

2.2 Variables, affectation, séquence

Définition 2.2 – *Variable et affectation*

Une **variable** est un emplacement mémoire identifié par un nom. L'instruction $A \leftarrow 5$ signifie « affecter la valeur 5 à A ». Attention : $A \leftarrow A+1$ remplace l'ancienne valeur de A par (ancienne valeur) + 1.

Exemple 2.1 – *Trace d'algorithme*

Exécuter : $A \leftarrow 3$, $B \leftarrow A+7$, $A \leftarrow A \times 2$

Étape	Instruction	A	B
1	$A \leftarrow 3$	3	–
2	$B \leftarrow A+7$	3	10
3	$A \leftarrow A \times 2$	6	10

2.3 Structures de contrôle

Définition 2.3 – *Structures de base*

- **Séquence** : les instructions s'exécutent dans l'ordre.
- **Alternative** (`Si ...Sinon`) : exécution conditionnelle.

- **Itération** (`Tant que` / `Pour`) : répétition.

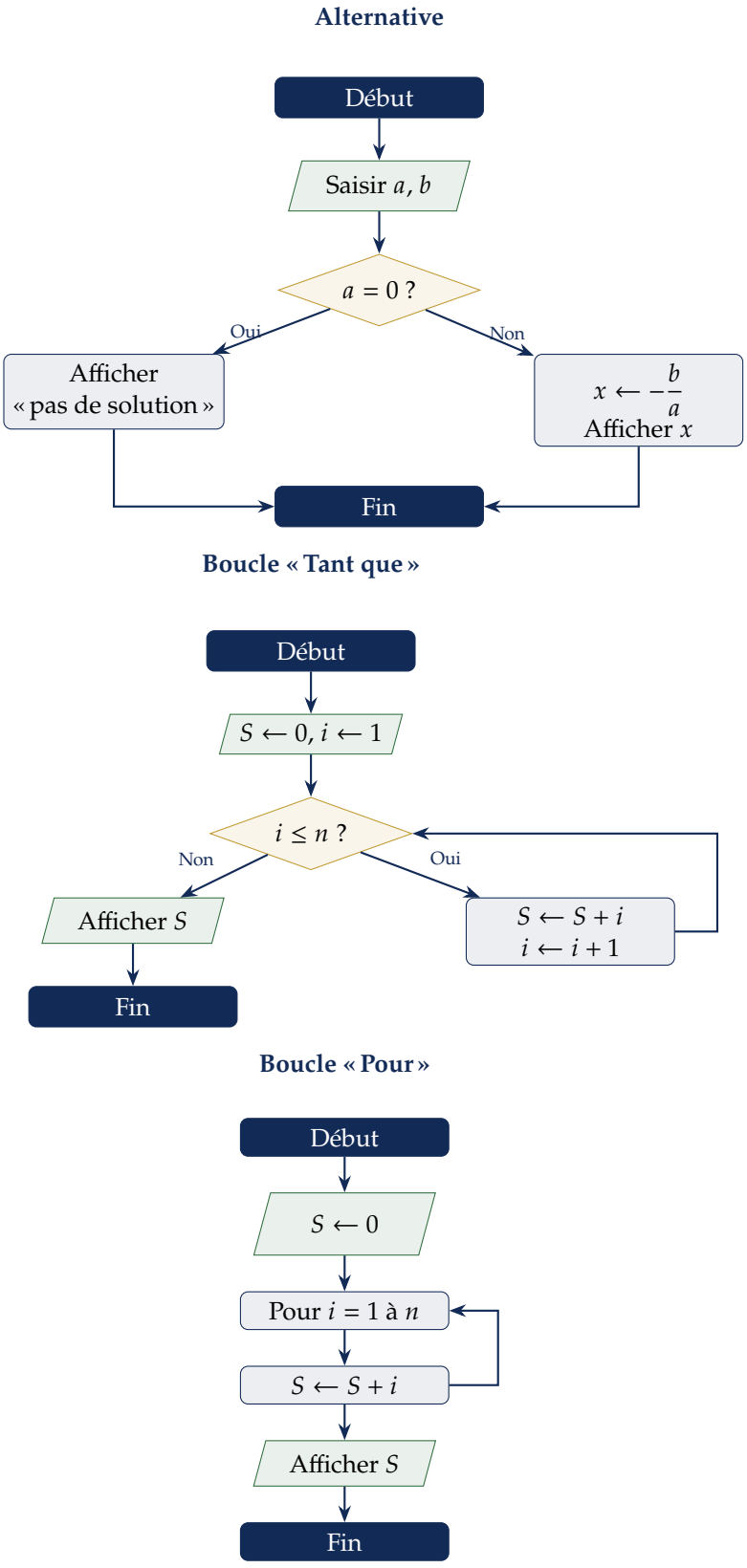


Figure 2.1: Les trois structures de contrôle en organigramme. **Alternative** : exécute l'une ou l'autre branche. **Tant que** : répète tant que la condition est vraie (test au début). **Pour** : répète un nombre fixé de fois.

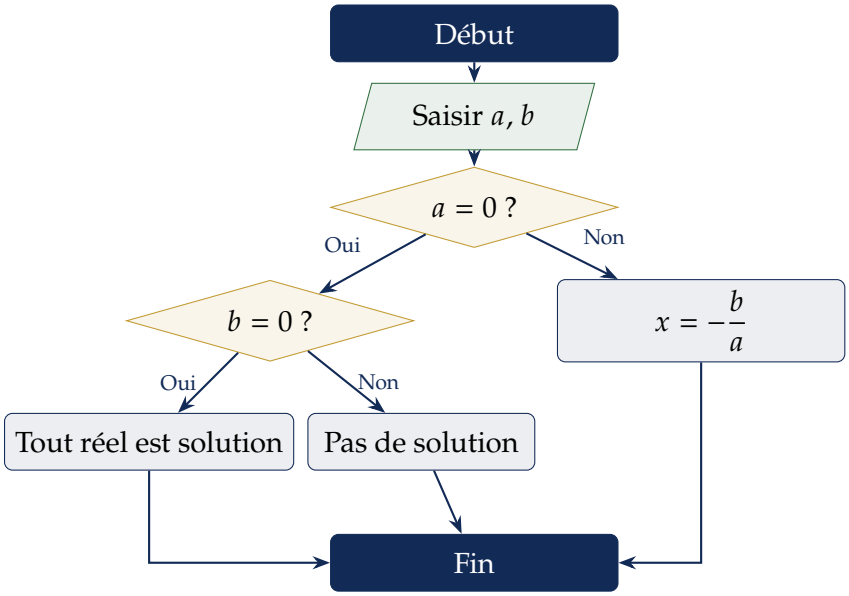


Figure 2.2: Organigramme : résolution de $ax + b = 0$.

Le même algorithme en pseudo-code :

Algorithme 2 – Résolution de $ax + b = 0$

Entrée : deux réels a et b
Sortie : la (ou les) solution(s) de $ax + b = 0$
Lire a, b;
Si $a = 0$ **alors**
  Si $b = 0$ **alors**
    Afficher « Tout réel est solution »;
  Sinon
    Afficher « Pas de solution »;
  Fin si
Sinon
  $x \leftarrow -b/a$;
  Afficher « Solution unique : $x =$ », x;
Fin si

Hors programme T10 – non exigible – Début d'un approfondissement

Tout le développement signalé ci-après est hors programme T10 : ne pas l'enseigner comme prérequis et ne pas l'évaluer. Cette partie propose des algorithmes plus longs qui combinent variables, conditions et boucles.

2.4 Méthodes et compléments de cours

Méthode 2.1 – *Construire la trace d'exécution*

Pour *exécuter à la main* un algorithme :

1. Créer un tableau avec une colonne par variable.
2. Exécuter chaque instruction dans l'ordre, ligne par ligne.
3. À chaque affectation, mettre à jour la colonne concernée en barrant l'ancienne valeur.
4. Pour un `Si/Sinon`, évaluer la condition *avant* de choisir la branche.
5. Pour un `Tant que`, répéter l'évaluation de la condition à chaque itération.

Méthode 2.2 – *Conventions de pseudo-code*

Dans ce manuel, on utilise les conventions suivantes :

- **Affectation :** $X \leftarrow$ expression
- **Lecture :** `Lire`(X) (saisie utilisateur)
- **Écriture :** `Afficher`(X)
- **Alternative :** `Si` *condition* `alors` … `Sinon` … `Fin Si`
- **Boucle bornée :** `Pour` i `de` a `à` b `faire` … `Fin Pour`
- **Boucle non bornée :** `Tant que` *condition* `faire` … `Fin Tant que`

Définition 2.4 – *Invariant de boucle*

Un **invariant de boucle** est une propriété vraie avant la boucle, conservée à chaque itération, et qui permet de prouver la correction de l'algorithme à la sortie.

Remarque. Le **coût** d'un algorithme se mesure en comptant le nombre d'opérations élémentaires effectuées en fonction de la taille n de l'entrée. Plus ce nombre croît lentement avec n, plus l'algorithme est efficace. On parle d'algorithme *rapide* (peu d'opérations) ou *lent* (beaucoup d'opérations).

ADA LOVELACE, PREMIÈRE PROGRAMMEUSE

En 1843, Ada Lovelace (1815–1852) publia ce qui est considéré comme le premier algorithme destiné à être exécuté par une machine : un programme pour calculer les nombres de Bernoulli sur la machine analytique *de Babbage, qui ne fut jamais construite. Elle imagina aussi que la machine pourrait composer de la musique, un siècle avant l'ordinateur.*

2.5 Exercices

★ Niveau 1 — Traces et écriture directe

Exercice 2.1 ALGO *Trace simple* ▷ *corrigé* ★

Donner la trace d'exécution et les valeurs finales de A, B, C :

$$A \leftarrow 5 \quad B \leftarrow 3 \quad C \leftarrow A + B \quad A \leftarrow A \times B \quad B \leftarrow C - A$$

Exercice 2.2 ALGO *Trace avec variables supplémentaires* ▷ *corrigé* ★

Donner la trace et les valeurs finales :

$$X \leftarrow 10 \quad Y \leftarrow 3 \quad Z \leftarrow X \div Y \quad R \leftarrow X - Z \times Y \quad X \leftarrow Z$$

(L'opérateur $\div$ désigne ici la division entière.)

Exercice 2.3 ALGO *Échange de deux variables* ▷ *corrigé* ★

On veut échanger les valeurs de A et B.

1. Montrer que le code suivant est incorrect : $A \leftarrow B$; $B \leftarrow A$.
2. Écrire un algorithme correct utilisant une variable temporaire T.
3. Donner la trace pour $A = 7$, $B = 3$.

Exercice 2.4 *Calcul de moyenne* ▷ *corrigé* ★

Écrire en pseudo-code puis implémenter Python un algorithme qui :

1. Saisit trois notes n_1, n_2, n_3 (entre 0 et 20).
2. Calcule et affiche leur moyenne.
3. Affiche « Reçu » si la moyenne est ≥ 10, « Recalé » sinon.

Exercice 2.5 *Équation du premier degré* ▸ *corrigé* ★

Écrire un algorithme qui saisit a et b, puis résout $ax + b = 0$ en gérant les trois cas : $a \neq 0$ (une solution), $a = 0$ et $b = 0$ (infinité de solutions), $a = 0$ et $b \neq 0$ (pas de solution).

Exercice 2.6 *Organigramme pair/impair* ▸ *corrigé* ★

1. Dessiner l'organigramme d'un algorithme qui saisit un entier n et affiche « pair » ou « impair ».
2. Écrire le pseudo-code correspondant.

Exercice 2.7 *Valeur absolue* ▸ *corrigé* ★

Écrire un algorithme qui saisit un réel x et affiche $|x|$ sans utiliser de fonction prédéfinie. Tester pour $x = -5$, $x = 0$, $x = 3{,}14$.

Exercice 2.8 *Maximum de trois nombres* ▸ *corrigé* ★

Écrire un algorithme qui saisit trois réels a, b, c et affiche le plus grand des trois.

1. Version avec des `Si` imbriqués.
2. Version en utilisant une variable `Max`.

Exercice 2.9 *Table de multiplication* ▸ *corrigé* ★

Écrire un algorithme qui saisit un entier $n \geq 1$ et affiche sa table de multiplication de $1 \times n$ à $10 \times n$. Donner la trace pour $n = 3$, puis implémenter en Python Python.

Exercice 2.10 *Lecture d'algorithme* ▸ *corrigé* ★

Que calcule et affiche cet algorithme ?

```
Lire N ;
S ← 0 ;
Pour i ← 1 à N faire
    S ← S + i ;
Fin pour
Afficher S ;
```

Vérifier pour $N = 5$.

Exercice 2.11 *Compter les multiples* ▷ *corrigé* ★

Écrire un algorithme qui affiche tous les multiples de 3 strictement inférieurs à 100. Combien y en a-t-il ?

Exercice 2.12 *Conversion Celsius/Fahrenheit* ▷ *corrigé* ★

La conversion est $F = \frac{9}{5}C + 32$. Écrire un algorithme qui affiche un tableau de conversion pour $C \in \{0, 10, 20, \ldots, 100\}$.

Exercice 2.13 *Puissance entière* ▷ *corrigé* ★

Écrire un algorithme qui saisit x et $n \geq 0$ et calcule x^n par multiplications successives (sans utiliser d'opérateur puissance). Tester pour $x = 2$, $n = 8$.

Exercice 2.14 *Somme des chiffres d'un entier* ▷ *corrigé* ★

Écrire un algorithme qui saisit un entier $n > 0$ et affiche la somme de ses chiffres. *Indication : utiliser la division euclidienne par* 10. Tester pour $n = 1234$.

Exercice 2.15 *Lecture de trace* ▷ *corrigé* ★

Exécuter cet algorithme pour $A = 4$, $B = 6$ et décrire ce qu'il calcule :

```
Entrée : deux entiers A, B > 0
Sortie : un entier
Lire A, B;
Tant que B ≠ 0 faire
    R ← A mod B                    // reste de la division de A par B
    A ← B;
    B ← R;
Fin tant que
Afficher A;
```

(Rappel : $A \mod B$ est le reste de la division de A par B.)

★★ Niveau 2 — Boucles et structures combinées

Exercice 2.16 *Suite arithmétique* ▷ *corrigé* ★★

Écrire un algorithme qui saisit u_0 et r, puis affiche les 20 premiers termes de la suite arithmétique $u_n = u_0 + nr$. Calculer aussi la somme $S = u_0 + u_1 + \cdots + u_{19}$ et vérifier avec la formule $S = 20u_0 + 190r$.

Exercice 2.17 *Suite géométrique et seuil* ▸ *corrigé* ★★

On considère la suite géométrique $u_n = 3 \times 2^n$. Écrire un algorithme puis l'implémenter Python pour :

1. Afficher les 10 premiers termes.
2. Trouver le plus petit n tel que $u_n > 1000$.

Vérifier algébriquement le résultat de (2).

Exercice 2.18 ALGO *Factorielle* ▸ *corrigé* ★★

Écrire un algorithme calculant $n!$ pour un entier $n \geq 0$ saisi.

1. Version itérative (`Pour`).
2. Ajouter une vérification : si $n < 0$, afficher un message d'erreur.
3. Combien de multiplications effectue l'algorithme ?

Exercice 2.19 *Maximum et position* ▸ *corrigé* ★★

Écrire un algorithme qui lit une séquence de n réels (un par un) et retourne :

- La valeur maximale.
- Sa position dans la séquence (rang du premier maximum trouvé).

Combien de comparaisons effectue-t-il ?

Exercice 2.20 *Recherche linéaire* ▸ *corrigé* ★★

Écrire un algorithme qui lit n entiers puis une valeur cible v, et affiche la position de la première occurrence de v (ou un message si v est absent).

1. Version avec `Tant que`.
2. Combien de comparaisons dans le pire cas ?

Exercice 2.21 *Palindrome* ▸ *corrigé* ★★

Un mot est un *palindrome* s'il se lit de la même façon dans les deux sens (« *kayak* », « *radar* », etc.).
Écrire un algorithme qui saisit un mot (une chaîne de caractères) et détermine si c'est un palindrome. *Indication : comparer le i-ème et le $(n-i)$-ème caractère.*

Exercice 2.22 *Suite de Syracuse* ▸ *corrigé* ★★

La suite de Syracuse part d'un entier $n > 0$:

$$u_{k+1} = \begin{cases} \dfrac{u_k}{2} & \text{si } u_k \text{ est pair,} \\ 3u_k + 1 & \text{si } u_k \text{ est impair.} \end{cases}$$

Écrire un algorithme qui :

1. Affiche tous les termes jusqu'à ce que $u_k = 1$.
2. Compte le nombre de termes (le *temps de vol*).
3. Calcule le maximum atteint (le *vol*).

Tester pour $n = 6$ et $n = 27$.

Exercice 2.23 *Diviseurs d'un entier* ▸ *corrigé* ★★

Écrire un algorithme qui saisit $n > 0$ et affiche tous ses diviseurs.

1. Version testant tous les entiers de 1 à n.
2. Version optimisée testant de 1 à $\lfloor \sqrt{n} \rfloor$ (et affichant les diviseurs par paires).
3. Comparer le nombre de tests dans les deux versions pour $n = 360$.

Exercice 2.24 ALGO *Nombre premier* ▸ *corrigé* ★★

Écrire un algorithme qui teste si un entier $n \geq 2$ est premier.

1. Justifier pourquoi il suffit de tester les diviseurs jusqu'à $\sqrt{n}$.
2. Écrire le pseudo-code.
3. Afficher tous les nombres premiers inférieurs à 100, puis implémenter en Python Python.

Exercice 2.25 *Zéro d'une fonction par dichotomie* Python ▸ *corrigé* ★★

On cherche à approcher une racine de $f(x) = x^3 - 2$ sur $[1, 2]$.

1. Vérifier que $f(1) < 0$ et $f(2) > 0$ (théorème des valeurs intermédiaires).
2. Écrire un algorithme de *dichotomie* : tant que $b - a > \varepsilon$, calculer $m = \dfrac{a+b}{2}$ et mettre à jour $[a, b]$.
3. Combien d'itérations pour $\varepsilon = 10^{-6}$?

4. Comparer avec $\sqrt[3]{2}$.

Exercice 2.26 *Codage de César* ▹ *corrigé* ★★

Le chiffrement de César décale chaque lettre de k rangs dans l'alphabet.

1. Écrire un algorithme qui chiffre un message lettre par lettre (on supposera les lettres codées de 0 à 25).
2. Écrire l'algorithme de déchiffrement.
3. Quel est le lien avec les calculs modulo 26 ?
4. Pourquoi ce chiffrement est-il considéré comme non sécurisé ?

Exercice 2.27 *Alignement de points* ▹ *corrigé* ★★

Écrire un algorithme qui saisit trois points $A(x_A, y_A)$, $B(x_B, y_B)$, $C(x_C, y_C)$ et détermine s'ils sont alignés. *Rappel : A, B, C sont alignés ssi $\overrightarrow{AB} \times \overrightarrow{AC} = 0$, i.e. $(x_B - x_A)(y_C - y_A) - (x_C - x_A)(y_B - y_A) = 0$.* Tester avec $A(0,0)$, $B(1,2)$, $C(2,4)$ et $A(0,0)$, $B(1,2)$, $C(2,5)$.

Exercice 2.28 *Calcul de π par série* ▹ *corrigé* ★★

La série de Leibniz donne : $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \cdots = \displaystyle\sum_{k=0}^{\infty} \frac{(-1)^k}{2k+1}$.

1. Écrire un algorithme qui calcule la somme des N premiers termes.
2. Afficher l'approximation de π pour $N = 10, 100, 1000, 10000$.
3. Cette série converge-t-elle vite ? Combien de termes pour avoir 5 décimales correctes ?

Exercice 2.29 *Schéma de Hörner* ▹ *corrigé* ★★

Évaluer le polynôme $P(x) = a_n x^n + \cdots + a_1 x + a_0$ en un point x_0 par le schéma de Hörner :

$$P(x_0) = (\ldots((a_n x_0 + a_{n-1})x_0 + a_{n-2})\ldots)x_0 + a_0.$$

1. Écrire l'algorithme (les coefficients sont lus de a_n à a_0).
2. Combien de multiplications effectue-t-il ? Comparer avec la méthode naïve.
3. Appliquer à $P(x) = 2x^3 - 5x^2 + 3x - 1$ en $x_0 = 2$.

Hors programme T10 – non exigible – Début d'un approfondissement

Tout le développement signalé ci-après est hors programme T10 : ne pas l'enseigner comme prérequis et ne pas l'évaluer. Les suites, la preuve par invariant, l'analyse du nombre d'opérations et les algorithmes spécialisés qui suivent sont facultatifs.

★★★ Niveau 3 — Raisonnement et algorithmes avancés

Exercice 2.30 ALGO *Suite de Fibonacci et rapport doré* Python ▷ *corrigé* ★★★

Écrire un algorithme calculant les N premiers termes de la suite de Fibonacci ($u_0 = 0$, $u_1 = 1$, $u_{n+1} = u_n + u_{n-1}$).

1. Afficher aussi le rapport $\dfrac{u_{n+1}}{u_n}$ pour $n \geq 1$.

2. Vers quelle valeur ce rapport semble-t-il converger ? (C'est le nombre d'or $\varphi = \dfrac{1+\sqrt{5}}{2}$.)

3. Modifier l'algorithme pour s'arrêter dès que ce rapport diffère de φ de moins de 10^{-6}.

Exercice 2.31 ALGO *Preuve de correction : invariant* ▷ *corrigé* ★★★

Considérons l'algorithme de calcul de $n!$:

Algorithme 5 – Calcul itératif de $n!$

Entrée : un entier $n \geq 0$
Sortie : $n!$
Lire n;
$F \leftarrow 1$;
$k \leftarrow 1$;
Tant que $k \leq n$ **faire**
  $F \leftarrow F \times k$;
  $k \leftarrow k + 1$;
Fin tant que
Afficher F;

1. Vérifier que $F = (k-1)!$ est un invariant de boucle.

2. En déduire la valeur de F à la sortie de la boucle.

3. L'algorithme est-il correct ? Termine-t-il toujours ?

Exercice 2.32 *Algorithme d'Euclide* ▷ *corrigé* ★★★

L'algorithme d'Euclide calcule $\text{pgcd}(a, b)$: tant que $b \neq 0$, poser $r \leftarrow a \mod b$, $a \leftarrow b$, $b \leftarrow r$.

1. Écrire le pseudo-code et donner la trace pour $a = 48$, $b = 18$.
2. Démontrer la correction : montrer que $\text{pgcd}(a, b) = \text{pgcd}(b, a \mod b)$.
3. Montrer que l'algorithme termine (le reste strictement décroît).
4. Combien d'étapes pour $a = F_{n+1}$, $b = F_n$ (nombres de Fibonacci) ?

Exercice 2.33 ALGO *Exponentiation rapide* ▷ *corrigé* ★★★

La méthode naïve calcule x^n en $n - 1$ multiplications. L'*exponentiation rapide* (squaring) utilise :

$$x^n = \begin{cases} 1 & n = 0, \\ (x^{n/2})^2 & n \text{ pair}, \\ x \cdot x^{n-1} & n \text{ impair}. \end{cases}$$

1. Écrire l'algorithme itératif correspondant.
2. Donner la trace pour $x = 2$, $n = 13$.
3. Combien de multiplications effectue-t-il en fonction de n ?
4. Comparer avec la méthode naïve pour $n = 1000$.

Exercice 2.34 ALGO *Tri bulles* ▷ *corrigé* ★★★

Le *tri à bulles* parcourt le tableau en comparant chaque paire d'éléments adjacents et en les échangeant si nécessaire, répétant jusqu'à ce que le tableau soit trié.

1. Écrire le pseudo-code du tri à bulles pour un tableau de n éléments.
2. Donner la trace pour $[5, 3, 8, 1, 9, 2]$.
3. Combien de comparaisons dans le pire cas ?
4. Proposer une optimisation : s'arrêter si aucun échange n'a eu lieu.

Exercice 2.35 ALGO *Recherche dichotomique* ▷ *corrigé* ★★★

Dans un tableau trié de n éléments, la *recherche dichotomique* compare la valeur cherchée avec l'élément du milieu et réduit le domaine de recherche de moitié à chaque étape.

1. Écrire le pseudo-code.

2. Donner la trace pour la recherche de 17 dans $[2, 5, 8, 12, 17, 23, 38]$.

3. Pour $n = 8$: combien d'itérations sont nécessaires ? Et pour $n = 16$, $n = 64$, $n = 1024$? Remplir le tableau :

n	Dichotomie	Linéaire (pire cas)
8		
16		
64		
1024		

4. Pour $n = 10^6$: estimer le nombre d'itérations de chaque méthode.

Exercice 2.36 *Codage et décodage binaire* ▸ *corrigé* ★★★

1. Écrire un algorithme qui convertit un entier $n \geq 0$ en binaire (afficher les bits de poids faible au poids fort en utilisant la division euclidienne par 2).

2. Écrire l'algorithme inverse (binaire vers décimal).

3. Tester pour $n = 42$.

4. Combien de bits faut-il pour représenter un entier n ?

Problèmes

Problème 2.1 – *Simulation d'un jeu de dés* ▸ *corrigé* ★

On simule le lancer de deux dés à 6 faces. Le score est la somme des deux résultats.

1. Écrire l'algorithme ci-dessous en pseudo-code, puis l'implémenter Python :

Algorithme 6 – Simulation de N lancers de dés

Entrée : un entier N (nombre de lancers)
Sortie : le nombre de fois où la somme vaut 7

```
compte ← 0;
Pour i ← 1 à N faire
    d1 ← Hasard(1, 6)                    // entier aléatoire entre 1 et 6
    d2 ← Hasard(1, 6);
    Si d1 + d2 = 7 alors
        compte ← compte + 1;
    Fin si
Fin pour
Afficher compte;
```

2. Calculer théoriquement la probabilité d'obtenir 7. Vérifier avec votre simulation pour $N = 1000$.
3. Modifier l'algorithme pour afficher la distribution complète des sommes possibles (de 2 à 12).

Problème 2.2 – *Suite de Collatz et conjecture* ▸ *corrigé* ★★

La suite de Collatz est $u_{n+1} = \frac{u_n}{2}$ si u_n pair, $3u_n + 1$ sinon. La conjecture (non prouvée !) affirme qu'elle atteint toujours 1.

1. Écrire un algorithme qui calcule le temps de vol (nombre de pas avant d'atteindre 1) pour tout n donné.
2. Trouver par ordinateur la valeur $n \leq 1000$ qui maximise le temps de vol. Quelle est cette valeur maximale ?
3. Pourquoi ne peut-on pas « prouver » la conjecture par cet algorithme, même en testant des millions de cas ?

Problème 2.3 – *Méthode de Héron : racine carrée* ▸ *corrigé* ★

La méthode de Héron approche $\sqrt{a}$ par la suite : $u_0 = a$, $\quad u_{n+1} = \dfrac{1}{2}\left(u_n + \dfrac{a}{u_n}\right)$.

1. Écrire l'algorithme suivant et le tester Python :

Algorithme 7 – Approximation de $\sqrt{a}$ par la méthode de Héron

Entrée : un réel $a > 0$, une précision $\varepsilon > 0$
Sortie : une approximation de $\sqrt{a}$
$u \leftarrow a$;
Répéter
  $u_{\text{new}} \leftarrow \frac{1}{2}\left(u + \frac{a}{u}\right)$;
  $u \leftarrow u_{\text{new}}$;
Jusqu'à $|u_{\text{new}} - u| < \varepsilon$
Afficher$u$;

2. Calculer à la main les 4 premiers termes pour $a = 2$. Comparer avec $\sqrt{2} \approx 1{,}41421$.
3. La suite converge-t-elle vite ? Compte le nombre d'itérations pour $\varepsilon = 10^{-6}$.
4. Démontrer algébriquement que si $u_n > 0$, alors $u_{n+1} \geq \sqrt{a}$. *(Indication : inégalité arithmético-géométrique $\frac{x+y}{2} \geq \sqrt{xy}$ pour $x, y > 0$.)*

Problème 2.4 – *Algorithme de Horner et évaluation de polynômes* ▹ *corrigé* ★★

On considère $P(x) = 3x^4 - 7x^3 + 2x^2 + x - 5$.

1. Calculer $P(2)$ en développant directement. Compter le nombre de multiplications effectuées.
2. Le schéma de Hörner réécrit : $P(x) = ((((3)x - 7)x + 2)x + 1)x - 5$. Calculer $P(2)$ en appliquant les parenthèses de l'intérieur vers l'extérieur. Compter les multiplications.

Algorithme 8 – Schéma de Hörner pour $P(x_0) = a_n x^n + \cdots + a_0$

Entrée : les coefficients $a_n, a_{n-1}, \ldots, a_0$ et un réel x_0
Sortie : $P(x_0)$
$val \leftarrow a_n$;
Pour $k \leftarrow n - 1$ **à**0 **faire**
  $val \leftarrow val \times x_0 + a_k$;
Fin pour
Afficherval;

3. Appliquer l'algorithme à la main pour $P(x) = 3x^4 - 7x^3 + 2x^2 + x - 5$, $x_0 = 2$. Vérifier que vous obtenez le même résultat qu'en (1).
4. Implémenter cet algorithme Python et tester sur $P(x) = x^3 - 6x^2 + 11x - 6$ pour $x_0 \in \{1, 2, 3\}$.

Problème 2.5 – *Cryptographie par décalage : attaque* ▹ *corrigé* ★★

Un message est chiffré par décalage de César avec une clé k inconnue.

1. Écrire un algorithme qui essaie toutes les clés de 0 à 25 et affiche les 26 déchiffrements possibles.
2. Comment identifier automatiquement le bon déchiffrement ? *(Indication : compter la lettre « e », la plus fréquente en français.)*
3. Cet algorithme d'attaque est dit *force brute*. Pourquoi est-il inapplicable contre des chiffrements modernes ?

Problème 2.6 – *Jeu du plus/moins* ▹ *corrigé* ★

L'ordinateur choisit un entier aléatoire n entre 1 et 100. L'utilisateur propose des valeurs ; l'ordinateur répond « plus », « moins » ou « trouvé ».

1. Écrire l'algorithme du jeu.
2. Quelle stratégie minimise le nombre de tentatives ?
3. Montrer que la stratégie optimale (dichotomie) réussit en au plus $\lceil \log_2 100 \rceil = 7$ tentatives.
4. Modifier l'algorithme pour compter les tentatives.

Problème 2.7 – *Crible d'Ératosthène* ▹ *corrigé* ★★

Le *crible d'Ératosthène* liste tous les nombres premiers jusqu'à N : on initialise un tableau de booléens à **V**, puis pour chaque premier p, on marque à **F** ses multiples $p^2, p^2 + p, p^2 + 2p, \ldots$

1. Écrire l'algorithme en pseudo-code.
2. Donner la trace pour $N = 30$.
3. Pour $N = 100$: compter le nombre total de cases marquées pendant l'exécution. L'algorithme est-il rapide ?
4. Pourquoi commence-t-on le marquage à p^2 plutôt qu'à $2p$?

Problème 2.8 – *Tri fusion : principe* ▹ *corrigé* ★★★

Le *tri fusion* divise le tableau en deux moitiés, les trie récursivement, puis les fusionne.

1. Écrire l'algorithme de *fusion* de deux tableaux triés.
2. Écrire le schéma récursif du tri fusion (on peut décrire la récursion sans la coder entièrement).
3. Appliquer sur $[5, 3, 8, 1, 9, 2, 4, 7]$ en détaillant les étapes.
4. Pour $n = 8$ éléments, compter le nombre total de comparaisons. Le tri fusion est-il plus rapide que le tri à bulles sur cet exemple ?

Problème 2.9 – *Chiffrement de Vigenère* ▸ *corrigé* ★★

Le chiffrement de Vigenère est une extension du chiffre de César : on utilise une clé de plusieurs lettres. Si la clé est « CLÉ » (rang 2, 11, 4 dans l'alphabet), on décale le i-ème caractère du message par le i-ème rang de la clé (cycliquement).

1. Chiffrer le message « BONJOUR » avec la clé « MATHS ». *(A=0, B=1, . . . , Z=25 ; appliquer le décalage modulo 26.)*
2. Écrire l'algorithme de chiffrement Python.
3. Écrire l'algorithme de déchiffrement.
4. Pourquoi ce chiffrement est-il plus sûr que le chiffre de César seul ? Combien de clés de longueur 5 existe-t-il ?

Problème 2.10 – *Algorithme de compression RLE* ▸ *corrigé* ★★

Le *Run-Length Encoding* (RLE) compresse une séquence en remplaçant les suites de caractères identiques par le caractère et son nombre de répétitions (ex. : `AAABBBBA` → `3A4B1A`).

1. Écrire l'algorithme de compression.
2. Écrire l'algorithme de décompression.
3. Tester sur `AAAAAABBBCCCCCD`.
4. Dans quels cas ce codage est-il inefficace ?

Problème 2.11 – L&A *Correction logique d'un algorithme* ▸ *corrigé* ★★

Voici un algorithme censé calculer le maximum d'une liste de n valeurs :

Algorithme 9 – Maximum (avec bug)

$max \leftarrow 0$;
Pour $i \leftarrow 1$ **à** n **faire**
  Si $L[i] > max$ **alors**
    $max \leftarrow L[i]$;
  Fin si
Fin pour
Afficher max;

1. Tester avec $L = \{3, 7, 2, 9, 5\}$: donne-t-il le bon résultat ?
2. Tester avec $L = \{-4, -1, -7, -2\}$: est-ce correct ? Si non, identifier logiquement l'erreur.
3. La proposition « l'algorithme calcule correctement le maximum pour toute liste » est-elle vraie ou fausse ? Donner un contre-exemple si fausse.
4. Corriger l'algorithme pour qu'il soit correct pour n'importe quelle liste.
5. Énoncer sous forme logique l'invariant correct : quelle propriété `max` doit-il vérifier après chaque itération ?

Problème 2.12 – *Calcul de π par méthode de Monte-Carlo* ▹ *corrigé* ★★

On tire aléatoirement des points (x, y) dans $[0, 1]^2$. Un point est dans le quart de disque unité si $x^2 + y^2 \leq 1$. La proportion de tels points approche $\pi/4$.

1. Écrire l'algorithme simulant N tirages et estimant π.
2. Tester pour $N = 100, 1000, 10^6$.
3. Estimer l'erreur en fonction de N.
4. Pourquoi appelle-t-on cela la *méthode de Monte-Carlo* ?

Problème 2.13 – *Statistiques par algorithme* Python ▹ *corrigé* ★★

Écrire un programme Python qui :

1. Lit une liste de n notes (entre 0 et 20) saisies par l'utilisateur.
2. Calcule et affiche la moyenne, le minimum, le maximum et l'étendue.
3. Calcule et affiche la médiane (après tri de la liste).

4. Trace un histogramme des notes regroupées par tranches : $[0, 5[$, $[5, 10[$, $[10, 15[$, $[15, 20]$.
5. Bonus : afficher si la classe a une moyenne ≥ 10 ou < 10.

On pourra utiliser les fonctions `sorted()`, `len()`, *et la bibliothèque* `matplotlib` *pour le tracé.*

Défis

★ **Défi – hors programme T10, non exigible** – ***Tri par sélection***

Le *tri par sélection* cherche le minimum dans la partie non triée et le place au début.

1. Écrire le pseudo-code.
2. Donner la trace pour $[5, 3, 8, 1, 9, 2]$.
3. Compter les comparaisons et les échanges.
4. Compter le nombre de comparaisons lorsque la liste contient n valeurs, puis lorsque la liste est déjà triée.
5. Comparer avec le tri à bulles (exercice 2.31).

★ **Défi – hors programme T10, non exigible** – ***Récursivité : flocon de Koch***

Le *flocon de Koch* se construit ainsi : on part d'un segment de longueur L. À chaque étape, on remplace chaque segment par quatre segments, chacun de longueur $L/3$, en formant une « bosse » triangulaire.

1. Décrire en pseudo-code l'algorithme récursif qui affiche le nombre de segments et la longueur totale après n étapes. *(Paramètres : longueur initiale, profondeur n.)*
2. Compléter le tableau :

Étape n	Nombre de segments	Longueur totale
0	1	L
1	4	$4L/3$
2		
3		
n		

3. La longueur totale est-elle bornée quand $n \to +\infty$? Que se passe-t-il si on part d'un triangle équilatéral et qu'on applique la construction à chacun de ses côtés ?

4. Expliquer pourquoi un algorithme récursif sans condition d'arrêt serait problématique : que se passerait-il ?

★ Défi – hors programme T10, non exigible – *Problème des quatre reines*

Placer 4 reines sur un échiquier 4×4 sans qu'aucune ne menace une autre (pas deux sur la même ligne, la même colonne, ni la même diagonale).

1. Pourquoi est-il impossible de placer 2 reines sur un échiquier 2×2 ? Et 3 reines sur un 3×3 ?

2. Sur le 4×4, on place les reines une par colonne. La première reine est en colonne 1, ligne a. Pour chaque valeur de $a \in \{1, 2, 3, 4\}$, déterminer quelles lignes sont disponibles en colonne 2.

3. En continuant le raisonnement, trouver **à la main** toutes les solutions (il y en a exactement 2). Les dessiner sur des grilles 4×4.

4. Écrire en pseudo-code un algorithme qui essaie systématiquement toutes les positions possibles pour les 4 reines (une par colonne) et affiche les solutions valides. Combien d'essais au maximum ?

★ Défi – hors programme T10, non exigible – *Simulation d'un automate simple*

Un *distributeur de boissons* fonctionne comme suit : il accepte des pièces de 1 € ou 2 € et distribue une boisson quand le total atteint ou dépasse 3 €.

1. Écrire un algorithme qui simule ce distributeur : il lit des pièces une par une (valeur 1 ou 2) et affiche « Boisson distribuée ! » dès que la somme ≥ 3. Il restitue l'excédent éventuel.

2. Donner la trace pour la séquence de pièces $1, 1, 2$. Puis pour $2, 2$.

3. Modifier l'algorithme pour refuser les pièces qui ne sont ni 1 € ni 2 € (afficher « Pièce invalide »).

4. Combien existe-t-il de séquences de pièces différentes aboutissant à exactement 3 € (sans excédent) ? Les lister systématiquement.

5. **Extension :** le distributeur a un *état interne* (montant cumulé). Représenter les états possibles et les transitions entre eux par un diagramme (cercles = états, flèches = pièces).

3. **Déduction en chaîne :** $P \Rightarrow Q$, $Q \Rightarrow R$, P est vrai $\Rightarrow R$ est vrai. Par transitivité (exercice 1.17), $P \Rightarrow R$, puis modus ponens. □

4. Le **raisonnement par cas** est la règle : de $(P \Rightarrow R) \wedge (Q \Rightarrow R) \wedge (P \vee Q)$, conclure R. Il est valide : si $P \vee Q$ est vraie, l'un des deux cas s'applique, et dans les deux cas R est vrai.

Chapitre 2 — Algorithmique

Exercices

Exercice 2.1 – *Trace simple* ↑ énoncé

Instruction	A	B	C
$A \leftarrow 5$	5	—	—
$B \leftarrow 3$	5	3	—
$C \leftarrow A + B$	5	3	8
$A \leftarrow A \times B$	15	3	8
$B \leftarrow C - A$	15	-7	8

Valeurs finales : $A = 15$, $B = -7$, $C = 8$.

Exercice 2.2 – *Trace avec variables supplémentaires* ↑ énoncé

$X \leftarrow 10$, $Y \leftarrow 3$: division entière $Z \leftarrow X \div Y = \lfloor 10/3 \rfloor = 3$. Reste : $R \leftarrow X - Z \times Y = 10 - 9 = 1$. $X \leftarrow Z = 3$.

Valeurs finales : $X = 3$, $Y = 3$, $Z = 3$, $R = 1$. R est le reste de la division euclidienne de 10 par 3.

Exercice 2.3 – *Échange de deux variables* ↑ énoncé

1. Code incorrect : $A \leftarrow B$ écrase la valeur de A avant qu'elle soit sauvegardée. Ensuite $B \leftarrow A$ ne fait que copier B dans B.

2. Algorithme correct avec variable temporaire :

Algorithme 25 – Échange correct

$T \leftarrow A$;
$A \leftarrow B$;
$B \leftarrow T$;

3. Trace pour $A = 7$, $B = 3$: $T \leftarrow 7$; $A \leftarrow 3$; $B \leftarrow 7$. Résultat : $A = 3$, $B = 7$. Échange réussi. □

Exercice 2.4 – *Calcul de moyenne* ↑ énoncé

Algorithme 26 – Moyenne et résultat

Lire n_1, n_2, n_3 ;
$M \leftarrow (n_1 + n_2 + n_3)/3$;
Afficher « Moyenne : », M ;
Si $M \geq 10$ **alors**
    Afficher « Reçu » ;
Sinon
    Afficher « Recalé » ;
Fin si

Implémentation Python :

```python
n1, n2, n3 = float(input()), float(input()), float(input())
m = (n1 + n2 + n3) / 3
print(f"Moyenne : {m:.2f}")
print("Recu" if m ≥ 10 else "Recale")
```

Exercice 2.5 – *Équation du premier degré* ↑ énoncé

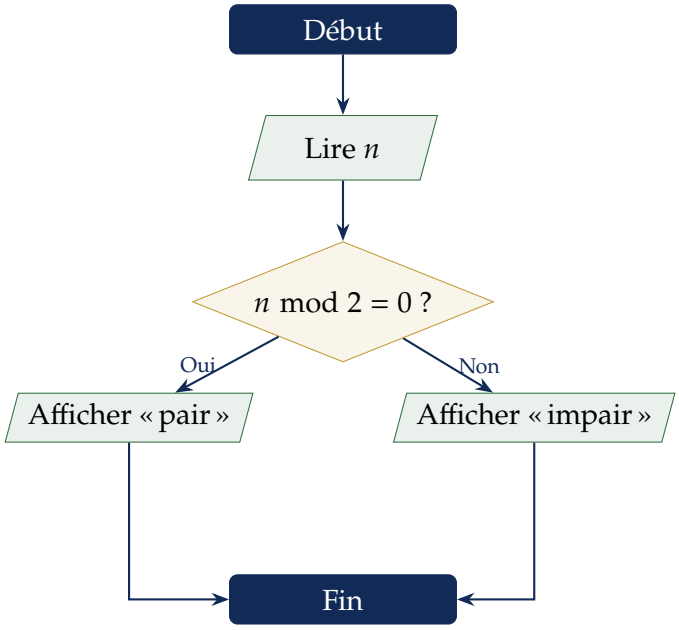
Algorithme 27 – Résolution de $ax + b = 0$

Lire a, b ;
Si $a \neq 0$ **alors**
    $x \leftarrow -b/a$;
    Afficher « Solution unique : $x =$ », x ;
Sinon si $b = 0$ **alors**
    Afficher « Tout réel est solution » ;
Sinon
    Afficher « Pas de solution » ;
Fin si

Exemples : $a = 2, b = -6 \Rightarrow x = 3$; $a = 0, b = 0 \Rightarrow$ infinité ; $a = 0, b = 5 \Rightarrow$ aucune.

Exercice 2.6 – *Organigramme pair/impair* ↑ énoncé

1. Organigramme :



2. Pseudo-code :

Algorithme 28 – Pair ou impair

Lire n;
Si n mod 2 = 0 **alors**
    Afficher « pair »;
Sinon
    Afficher « impair »;
Fin si

Exercice 2.7 – *Valeur absolue* ↑ énoncé

Algorithme 29 – Valeur absolue

Lire x;
Si $x \geq 0$ **alors**
    Afficher x;
Sinon
    Afficher $-x$;
Fin si

Tests : $x = -5 \Rightarrow 5$; $x = 0 \Rightarrow 0$; $x = 3{,}14 \Rightarrow 3{,}14$.

Exercice 2.8 – *Maximum de trois nombres* ↑ énoncé

(1) Si imbriqués :

Algorithme 30 – Maximum (Si imbriqués)

Lire a, b, c;
Si $a \geq b$ **alors**
  Si $a \geq c$ **alors**
    Afficher a;
  Sinon
    Afficher c;
  Fin si
Sinon
  Si $b \geq c$ **alors**
    Afficher b;
  Sinon
    Afficher c;
  Fin si
Fin si

(2) Variable Max :

Algorithme 31 – Maximum (variable Max)

Lire a, b, c;
$Max \leftarrow a$;
Si $b > Max$ **alors**
  $Max \leftarrow b$;
Fin si
Si $c > Max$ **alors**
  $Max \leftarrow c$;
Fin si
Afficher Max;

La deuxième version est plus généralisable : remplacer les deux `Si` par une boucle sur n éléments.

Exercice 2.9 – *Table de multiplication* ↑ énoncé

Trace pour $n = 3$: affiche $3, 6, 9, 12, 15, 18, 21, 24, 27, 30$.

Algorithme 32 – Table de n

Lire n;
Pour $k \leftarrow 1$ **à** 10 **faire**
  Afficher k, « × », n, « = », $k \times n$;
Fin pour

```python
n = int(input("n = "))
for k in range(1, 11):
    print(f"{k} x {n} = {k*n}")
```

Exercice 2.10 – *Lecture d'algorithme* ↑ énoncé

L'algorithme calcule $S = 1 + 2 + \cdots + N = \dfrac{N(N+1)}{2}$.
Vérification pour $N = 5$: $S = 15 = 5 \times 6/2$.✓
L'invariant de boucle est « après i itérations, $S = 1 + 2 + \cdots + i$ ».

Exercice 2.11 – *Compter les multiples* ↑ énoncé

Algorithme 33 – Multiples de 3 $<$ 100

compte $\leftarrow 0$;
Pour $k \leftarrow 3$ **à**99 **par pas de** 3 **faire**
  Afficher$k$;
  compte $\leftarrow$ *compte* $+ 1$;
Fin pour
Afficher« Total : », *compte*;

Il y en a $\lfloor 99/3 \rfloor = 33$ (de 3 à 99).

Exercice 2.12 – *Conversion Celsius/Fahrenheit* ↑ énoncé

Algorithme 34 – Tableau Celsius/Fahrenheit

Pour $C \leftarrow 0$ **à**100 **par pas de** 10 **faire**
  $F \leftarrow \frac{9}{5} \times C + 32$;
  Afficher$C$, « degrés Celsius = », F, « degrés Fahrenheit »;
Fin pour

Premières lignes : 0°C = 32°F ; 10°C = 50°F ; 100°C = 212°F.

Exercice 2.13 – *Puissance entière* ↑ énoncé

Algorithme 35 – x^n par multiplications

Lire$x$, n;
$P \leftarrow 1$;
Pour $k \leftarrow 1$ **à**n **faire**
  $P \leftarrow P \times x$;
Fin pour
Afficher$P$;

Trace pour $x = 2$, $n = 8$: P prend les valeurs $2, 4, 8, 16, 32, 64, 128, 256$. Valeur finale : $P = 256 = 2^8$.✓

Exercice 2.14 – *Somme des chiffres d'un entier* ↑ énoncé

Algorithme 36 – Somme des chiffres

Lire$n$;
$S \leftarrow 0$;

Tant que $n > 0$ **faire**
  $S \leftarrow S + (n \bmod 10)$;
  $n \leftarrow \lfloor n/10 \rfloor$;
Fin tant que
Afficher$S$;

Trace pour $n = 1234$: $S = 0 + 4 = 4$, $n = 123$; $S = 4 + 3 = 7$, $n = 12$; $S = 7 + 2 = 9$, $n = 1$; $S = 9 + 1 = 10$, $n = 0$; Fin : $S = 10 = 1 + 2 + 3 + 4$.✓

Exercice 2.15 – *Lecture de trace : algorithme d'Euclide* ↑ *énoncé*

Trace pour $A = 48$, $B = 18$:

A	B	$R = A \bmod B$
48	18	12
18	12	6
12	6	0
6	0	—

L'algorithme calcule le **PGCD**. Valeur finale de $A = 6 = \text{pgcd}(48, 18)$.

Exercice 2.16 – *Suite arithmétique* ↑ *énoncé*

Algorithme 37 – Suite arithmétique et somme

Lireu_0, r;
$u \leftarrow u_0$; $S \leftarrow 0$;
Pour $n \leftarrow 0$ **à**19 **faire**
  Afficher« $u_n =$ », u;
  $S \leftarrow S + u$;
  $u \leftarrow u + r$;
Fin pour
Afficher« Somme : », S;

Vérification : $S = 20u_0 + r(0 + 1 + \cdots + 19) = 20u_0 + 190r$. Par exemple pour $u_0 = 1$, $r = 2$: $S = 20 + 190 \times 2 = 400$. Formule : $20 \times 1 + 190 \times 2 = 20 + 380 = 400$.✓

Exercice 2.17 – *Suite géométrique et seuil* ↑ *énoncé*

Algorithme 38 – Suite géométrique $u_n = 3 \times 2^n$

$u \leftarrow 3$; $n \leftarrow 0$;
Pour $k \leftarrow 0$ **à**9 **faire**
  Afficher« $u_k =$ », u ; $u \leftarrow 2 \times u$;
Fin pour
$u \leftarrow 3$; $n \leftarrow 0$;
Tant que $u \leq 1000$ **faire**
  $u \leftarrow 2 \times u$; $n \leftarrow n + 1$;
Fin tant que

Afficher« Premier n tel que $u_n > 1000$: », n;

Vérification algébrique : $3 \times 2^n > 1000 \Rightarrow 2^n > 333{,}3 \Rightarrow n > \log_2(333{,}3) \approx 8{,}38$. Donc $n = 9$: $u_9 = 3 \times 512 = 1536 > 1000$.✓

Exercice 2.18 – *Factorielle* ↑ énoncé

Algorithme 39 – Factorielle

Lire$n$;
Si $n < 0$ **alors**
  Afficher« Erreur : n doit être ≥ 0 » ; **Stop**;
Fin si
$F \leftarrow 1$;
Pour $k \leftarrow 1$ **à**n **faire**
  $F \leftarrow F \times k$;
Fin pour
Afficher$F$;

1. Version itérative : voir ci-dessus.
2. Si $n < 0$, on affiche un message et on arrête.
3. Nombre de multiplications : $n - 1$ (pour $n \geq 1$; 0 pour $n = 0$).

Exemples : 5! = 120 ; 0! = 1 ; 10! = 3628800.

Exercice 2.19 – *Maximum et position* ↑ énoncé

Algorithme 40 – Maximum et sa position

Lire$n$;
Lirex_1 ; $M \leftarrow x_1$; $pos \leftarrow 1$;
Pour $i \leftarrow 2$ **à**n **faire**
  Lirex_i;
  Si $x_i > M$ **alors**
    $M \leftarrow x_i$; $pos \leftarrow i$;
  Fin si
Fin pour
Afficher« Max : », M, « en position : », pos;

Nombre de comparaisons : exactement $n - 1$ (une par élément après le premier).

Exercice 2.20 – *Recherche linéaire* ↑ énoncé

Algorithme 41 – Recherche linéaire

Liren, v ; $i \leftarrow 1$; **Vrai**trouvé $\leftarrow$ **Faux**;
Tant que $i \leq n$ **et non** trouvé **faire**
  Lirex_i;

```
    Si x_i = v alors
        trouvé ← Vrai;
    Sinon
        i ← i + 1;
    Fin si
Fin tant que
Si trouvé alors
    Afficher« Trouvé en position : », i;
Sinon
    Afficher« v absent »;
Fin si
```

1. Version `Tant que` : voir ci-dessus.
2. Pire cas : v absent, on fait n comparaisons.

Exercice 2.21 – *Palindrome* ↑ énoncé

Algorithme 42 – Test palindrome

```
Lire s ;  n ← longueur(s) ;  pal ← Vrai;
Pour i ← 1 à⌊n/2⌋ faire
    Si s[i] ≠ s[n + 1 − i] alors
        pal ← Faux;
    Fin si
Fin pour
Si pal alors
    Afficher« Palindrome »;
Sinon
    Afficher« Pas un palindrome »;
Fin si
```

Tests : `radar` : $r = r, a = a, d = d \Rightarrow$ palindrome. `bonjour` : $b \neq r \Rightarrow$ non palindrome.

Exercice 2.22 – *Suite de Syracuse* ↑ énoncé

Algorithme 43 – Suite de Syracuse

```
Lire n ;  k ← 0 ;  max ← n;
Tant que n ≠ 1 faire
    Si n mod 2 = 0 alors
        n ← n/2;
    Sinon
        n ← 3n + 1;
    Fin si
    k ← k + 1;
    Si n > max alors
        max ← n;
    Fin si
Fin tant que
```

Afficher« Temps de vol : », k ; **Afficher**« Altitude max : », max;

Pour $n = 6$: $6 \to 3 \to 10 \to 5 \to 16 \to 8 \to 4 \to 2 \to 1$: temps de vol 8, altitude max 16.
La conjecture de Collatz (non démontrée) affirme que toute suite atteint 1.

Exercice 2.23 – *Diviseurs d'un entier* ↑ énoncé

Version 1 :

Algorithme 44 – Diviseurs (version naïve)

Lire$n$;
Pour $k \leftarrow 1$ **à**n **faire**
  Si $n \bmod k = 0$ **alors**
    Afficher$k$;
  Fin si
Fin pour

n opérations.

Version 2 (optimisée) :

Algorithme 45 – Diviseurs (version optimisée)

Lire$n$;
Pour $k \leftarrow 1$ **à**$\lfloor\sqrt{n}\rfloor$ **faire**
  Si $n \bmod k = 0$ **alors**
    Afficher$k$;
    Si $k \neq n/k$ **alors**
      Affichern/k;
    Fin si
  Fin si
Fin pour

$\lfloor\sqrt{n}\rfloor$ opérations : beaucoup moins pour les grands n.
Pour $n = 36$: diviseurs $1, 2, 3, 4, 6, 9, 12, 18, 36$.

Exercice 2.24 – *Nombre premier* ↑ énoncé

1. Si n admet un diviseur $d > \sqrt{n}$, alors $n/d < \sqrt{n}$ est aussi un diviseur. Donc il suffit de tester jusqu'à $\sqrt{n}$.
2. Pseudo-code :

Algorithme 46 – Test de primalité

Lire$n$;
Si $n < 2$ **alors**
  Afficher« Non premier » ; **Stop**;
Fin si
$premier \leftarrow$ **Vrai**;
$k \leftarrow 2$;

```
Tant que k ≤ √n et premier faire
    Si n mod k = 0 alors
        premier ← Faux;
    Fin si
    k ← k + 1;
Fin tant que
Si premier alors
    Afficher« Premier »;
Sinon
    Afficher« Non premier »;
Fin si
```

3. Premiers < 100 : $2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97$. Il y en a 25.

Exercice 2.25 – *Zéro d'une fonction par dichotomie* ↑ *énoncé*

1. $f(1) = 1 - 2 = -1 < 0$ et $f(2) = 8 - 2 = 6 > 0$.✓ (TVI garantit une racine dans $]1, 2[$.)
2. Algorithme :

Algorithme 47 – Dichotomie pour $f(x) = x^3 - 2$

```
a ← 1 ;  b ← 2 ;  ε ← 10^−6;
Tant que b − a > ε faire
    m ← (a+b)/2;
    Si f(m) × f(a) ≤ 0 alors
        b ← m;
    Sinon
        a ← m;
    Fin si
Fin tant que
Afficher« Racine ≈ », (a+b)/2;
```

3. Nombre d'itérations : $n \geq \log_2((b-a)/\varepsilon) = \log_2(10^6) \approx 20$.
4. Limite : $\sqrt[3]{2} \approx 1{,}25992$. Vérification : $1{,}25992^3 \approx 2$.✓

Exercice 2.26 – *Codage de César* ↑ *énoncé*

Algorithme 48 – Chiffrement de César

```
Lirek ; (la clé de décalage);
Liremessage;
Pour chaque lettre c de message faire
    rang ← (rang(c) + k) mod 26;
    Afficherla lettre de rang rang;
Fin pour
```

Déchiffrement : remplacer $+k$ par $-k$ (ou $+26 - k$).

La clé $k = 13$ donne ROT13 : son propre inverse car $13 + 13 = 26 \equiv 0$. Le nombre de clés possibles est 26 : force brute triviale (essayer toutes).

Exercice 2.27 – *Alignement de points* ↑ énoncé

Algorithme 49 – Test d'alignement

Lire $x_A, y_A, x_B, y_B, x_C, y_C$;
$d \leftarrow (x_B - x_A) \times (y_C - y_A) - (x_C - x_A) \times (y_B - y_A)$;
Si $d = 0$ **alors**
    Afficher « Alignés » ;
Sinon
    Afficher « Non alignés » ;
Fin si

Test $A(0,0)$, $B(1,2)$, $C(2,4)$: $d = (1)(4) - (2)(2) = 4 - 4 = 0$: alignés.✓
Test $A(0,0)$, $B(1,2)$, $C(3,5)$: $d = (1)(5) - (3)(2) = 5 - 6 = -1 \neq 0$: non alignés.✓

Exercice 2.28 – *Calcul de π par série de Leibniz* ↑ énoncé

Algorithme 50 – π par Leibniz

Lire N ; $S \leftarrow 0$;
Pour $k \leftarrow 0$ **à** $N - 1$ **faire**
    $S \leftarrow S + (-1)^k/(2k+1)$;
Fin pour
Afficher $4 \times S$;

N	$4S$	Erreur
10	3,0418…	10^{-1}
100	3,1316…	10^{-2}
1 000	3,1406…	10^{-3}
10 000	3,1415…	10^{-4}

La convergence est lente : multiplier le nombre de termes par 10 ne gagne qu'environ une décimale. La série de Machin converge beaucoup plus vite.

Exercice 2.29 – *Schéma de Hörner* ↑ énoncé

Algorithme 51 – Schéma de Hörner

Lire n, x_0 ; **Lire** a_n ; $P \leftarrow a_n$;
Pour $k \leftarrow n - 1$ **à** 0 **(décroissant) faire**
    Lire a_k ; $P \leftarrow P \times x_0 + a_k$;
Fin pour
Afficher P ;

1. L'algorithme ci-dessus lit les coefficients de a_n à a_0.
2. Il effectue n multiplications (une par itération), contre $n(n+1)/2$ si chaque puissance x_0^k est recalculée séparément par multiplications successives.

3. Pour $P(x) = 2x^3 - 5x^2 + 3x - 1$ en $x_0 = 2$: Hörner : $((2\times 2 - 5)\times 2 + 3)\times 2 - 1 = (-1\times 2 + 3)\times 2 - 1 = 1 \times 2 - 1 = 1$. Vérification : $2(8) - 5(4) + 3(2) - 1 = 16 - 20 + 6 - 1 = 1$.✓

Exercice 2.30 – *Suite de Fibonacci et nombre d'or* ↑ *énoncé*

Algorithme 52 – Fibonacci et nombre d'or

$u \leftarrow 0$; $v \leftarrow 1$;
Pour $n \leftarrow 1$ **à** N **faire**
  Afficher « $F_n =$ », v, « rapport F_{n+1}/F_n : », $(u+v)/v$;
  $(u, v) \leftarrow (v, u+v)$;
Fin pour

1. Premiers termes (en commençant à $F_1 = 1$) : $1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, \ldots$

2. Rapports F_{n+1}/F_n : $1, 2, 1{,}5, 1{,}667, 1{,}6, 1{,}625, 1{,}615, \ldots$ Limite : $\varphi = \dfrac{1+\sqrt{5}}{2} \approx 1{,}61803\ldots$ (nombre d'or).

3. Pour s'arrêter dès que le rapport est assez proche de φ, remplacer la limite N par une boucle qui calcule $(u+v)/v$ et s'arrête quand $\left|(u+v)/v - \varphi\right| < 10^{-6}$. La mise à jour $(u, v) \leftarrow (v, u+v)$ n'est faite que si le seuil n'est pas encore atteint.

Exercice 2.31 – *Preuve de correction : invariant* ↑ *énoncé*

Invariant : au début de chaque itération avec valeur k, $F = (k-1)!$.

Initialisation : avant la boucle, $F = 1 = 0! = (1-1)!$. ✓

Conservation : si $F = (k-1)!$ en entrant dans l'itération, alors après $F \leftarrow F \times k$, on a $F = k! = (k+1-1)!$. ✓

Terminaison : la boucle s'arrête quand $k > n$, soit $k = n+1$. À ce moment, l'invariant donne $F = (n+1-1)! = n!$. □

Exercice 2.32 – *Algorithme d'Euclide* ↑ *énoncé*

1. Pseudo-code et trace :

a	b	$r = a \bmod b$
48	18	12
18	12	6
12	6	0

Résultat : $\text{pgcd}(48, 18) = 6$. Le pseudo-code est :

Algorithme 53 – PGCD par l'algorithme d'Euclide

Lire a, b;
Tant que $b \neq 0$ **faire**
  $r \leftarrow a \bmod b$; $a \leftarrow b$; $b \leftarrow r$;
Fin tant que

Afficher a;

2. **Correction :** montrons que $\text{pgcd}(a,b) = \text{pgcd}(b,r)$ où $r = a \bmod b$. Tout diviseur commun de a et b divise $r = a - qb$, donc divise b et r. Réciproquement, tout diviseur commun de b et r divise $a = qb + r$. L'ensemble des diviseurs communs est identique : $\text{pgcd}(a,b) = \text{pgcd}(b,r)$.□

3. À chaque étape, le second nombre diminue strictement; l'algorithme finit donc par atteindre un reste nul.

4. Pour $(a,b) = (F_{n+1}, F_n)$, avec $F_1 = F_2 = 1$ et $n \geq 2$, l'algorithme effectue $n-1$ divisions : pour $n \geq 3$, les restes sont $F_{n-1}, F_{n-2}, \ldots, F_2, 0$; pour $n = 2$, la seule division donne le reste 0.

Exercice 2.33 – *Exponentiation rapide* ↑ énoncé

Algorithme 54 – Exponentiation rapide (itérative)

Lire x, n ; $P \leftarrow 1$; $b \leftarrow x$;
Tant que $n > 0$ **faire**
  Si $n \bmod 2 = 1$ **alors**
    $P \leftarrow P \times b$;
  Fin si
  $n \leftarrow \lfloor n/2 \rfloor$;
  Si $n > 0$ **alors**
    $b \leftarrow b \times b$;
  Fin si
Fin tant que
Afficher P;

1. Version itérative : voir ci-dessus.

2. Pour $x = 2$, $n = 13$, les ètats (P, b, n) après chaque itération sont $(2,4,6)$, $(2,16,3)$, $(32,256,1)$, $(8192,256,0)$. On trouve bien $2^{13} = 8192$.

3. Pour $n \geq 1$, le nombre de multiplications est $\lfloor \log_2 n \rfloor$ mises au carré plus le nombre de chiffres 1 dans l'écriture binaire de n; pour $13 = 1101_2$, cela fait $3 + 3 = 6$.

4. Pour $n = 1000 = 1111101000_2$, cela donne $9 + 6 = 15$ multiplications, contre 999 par la méthode naïve.

Exercice 2.34 – *Tri à bulles* ↑ énoncé

Algorithme 55 – Tri à bulles

Entrée : Tableau T de n éléments
Pour $i \leftarrow 1$ **à** $n-1$ **faire**
  Pour $j \leftarrow 1$ **à** $n-i$ **faire**
    Si $T[j] > T[j+1]$ **alors**
      Échanger $T[j]$ et $T[j+1]$;
    Fin si
  Fin pour
Fin pour

1. Pseudo-code : voir ci-dessus.
2. Trace pour $T = [5,3,8,1,9,2]$: **Passe 1** : $[3,5,1,8,2,9]$; **Passe 2** : $[3,1,5,2,8,9]$; **Passe 3** : $[1,3,2,5,8,9]$; **Passe 4** : $[1,2,3,5,8,9]$; **Passe 5** : aucun changement.
3. Nombre de comparaisons : $(n-1)+(n-2)+\cdots+1 = \dfrac{n(n-1)}{2}$.
4. Optimisation : si aucun échange ne se produit lors d'une passe, le tableau est trié : arrêter.

Exercice 2.35 – *Recherche dichotomique* ↑ *énoncé*

Algorithme 56 – Recherche dichotomique

Entrée : Tableau T trié, valeur v
$g \leftarrow 1$; $d \leftarrow n$;
Tant que $g \leq d$ **faire**
  $m \leftarrow \lfloor (g+d)/2 \rfloor$;
  Si $T[m] = v$ **alors**
    Afficher« Trouvé en position : », m ; **Stop**;
  Sinon si $T[m] < v$ **alors**
    $g \leftarrow m+1$;
  Sinon
    $d \leftarrow m-1$;
  Fin si
Fin tant que
Afficher« v absent »;

1. Pseudo-code : voir ci-dessus.
2. Pour $T = [2,5,8,12,17,23,38]$ et $v = 17$: $g = 1, d = 7, m = 4$, $T[4] = 12 < 17$; puis $g = 5, d = 7, m = 6$, $T[6] = 23 > 17$; enfin $g = 5, d = 5, m = 5$, $T[5] = 17$.
3. Dans le pire cas (y compris une valeur absente), on obtient :

n	Dichotomie	Linéaire
8	4	8
16	5	16
64	7	64
1024	11	1024

4. Pour $n = 10^6$, au plus 20 tests suffisent par dichotomie, contre jusqu'à un million par recherche linéaire.

Exercice 2.36 – *Codage et décodage binaire* ↑ *énoncé*

Algorithme 57 – Décimal vers binaire

Lire$n$;
Si $n = 0$ **alors**
  Afficher« 0 » ; **Stop**;

Fin si
$bits \leftarrow$ chaîne vide;
Tant que $n > 0$ **faire**
  $bits \leftarrow (n \bmod 2) + bits$;
  $n \leftarrow \lfloor n/2 \rfloor$;
Fin tant que
Afficher$bits$;

Algorithme 58 – Binaire vers décimal

Lireune chaîne de bits $b_1 b_2 \ldots b_k$; $N \leftarrow 0$;
Pour $i \leftarrow 1$ **à**k **faire**
  $N \leftarrow 2 \times N + b_i$;
Fin pour
Afficher$N$;

Pour $n = 42$: $42 = 32 + 8 + 2 = 101010_2$. Vérification : $1 \cdot 32 + 0 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 42$.✓

Problèmes

Problème 2.1 – *Simulation d'un jeu de dés* ↑ *énoncé*

(1–2) L'algorithme de l'énoncé compte les sommes égales à 7. Pour deux dés équilibrés lancés indépendamment, les 36 couples ordonnés sont équiprobables. Les six couples favorables sont $(1,6), (2,5), (3,4), (4,3), (5,2), (6,1)$: la probabilité vaut $6/36 = 1/6$. Pour $N = 1000$, la fréquence simulée est souvent proche de $1/6$, mais sa valeur exacte varie d'une simulation à l'autre.

(3) Pour obtenir toutes les sommes, on conserve onze compteurs :

```python
import random
N = 1000
comptes = {s: 0 for s in range(2, 13)}
for _ in range(N):
    d1 = random.randint(1, 6)
    d2 = random.randint(1, 6)
    comptes[d1+d2] += 1
print("Somme 7:", comptes[7], "frequence:", comptes[7]/N)
for s in range(2, 13):
    print(s, comptes[s], comptes[s]/N)
```

Les effectifs théoriques parmi les 36 couples sont $1, 2, 3, 4, 5, 6, 5, 4, 3, 2, 1$ pour les sommes de 2 à 12.

Problème 2.2 – *Suite de Collatz et conjecture* ↑ *énoncé*

1. Algorithme :

Algorithme 59 – Temps de vol de Collatz

Lire$n$; $t \leftarrow 0$;
Tant que $n \neq 1$ **faire**
  Si $n \bmod 2 = 0$ **alors**

```
        n ← n/2;
    Sinon
        n ← 3n + 1;
    Fin si
    t ← t + 1;
Fin tant que
Afficher t;
```

2. Exemples : $n = 6 \to$ temps 8 ; $n = 27 \to$ temps 111.

3. Pour trouver le maximum sur $1..N$, on boucle sur toutes les valeurs. L'entier $n = 27$ a un temps de vol de 111 pour des n jusqu'à 100.

4. La conjecture est non démontrée : on ne sait pas si *tous* les entiers atteignent 1 en temps fini.

Problème 2.3 – *Méthode de Héron : racine carrée* ↑ énoncé

1. Pseudo-code :

Algorithme 60 – Méthode de Héron

```
Lire a, ε ;   u ← a;
Tant que |u − a/u| > ε faire
    u ← (u + a/u)/2;
Fin tant que
Afficher u;
```

2. Trace pour $a = 2$, $\varepsilon = 10^{-4}$: $u_0 = 2$, $u_1 = 1{,}5$, $u_2 = 1{,}4167$, $u_3 = 1{,}4142\ldots \approx \sqrt{2}$. Convergence en 4 itérations.

3. La convergence est **quadratique** : le nombre de décimales correctes double à chaque itération. C'est bien plus rapide que la dichotomie.

Problème 2.4 – *Algorithme de Hörner et évaluation de polynômes* ↑ énoncé

$P(x) = 3x^4 - 7x^3 + 2x^2 + x - 5$.

1. Naïvement : $P(2) = 3(16) - 7(8) + 2(4) + 2 - 5 = 48 - 56 + 8 + 2 - 5 = -3$. Nombre de multiplications : $4 + 3 + 2 + 1 = 10$ (en développant les puissances).

2. Hörner : $((((3)\times 2 - 7)\times 2 + 2)\times 2 + 1)\times 2 - 5 = (((-1)\times 2 + 2)\times 2 + 1)\times 2 - 5 = ((0)\times 2 + 1)\times 2 - 5 = 1 \times 2 - 5 = -3$.✓Nombre de multiplications : 4 : un par degré.

3. Hörner utilise une multiplication par degré, ce qui évite de recalculer séparément toutes les puissances de x.

Problème 2.5 – *Cryptographie par décalage : attaque* ↑ énoncé

1. Force brute (essayer les 26 clés) :

Algorithme 61 – Attaque par force brute

Lire*message*;
Pour $k \leftarrow 0$ **à**25 **faire**
  Afficher« Clé », k, « : », déchiffrer(*message*, k);
Fin pour

2. Identification automatique : une approche consiste à calculer la fréquence des lettres du texte déchiffré et de la comparer aux fréquences du français (la lettre la plus fréquente est « E » : 17%). La clé correcte maximise la correspondance.

3. Le chiffrement de César a une sécurité nulle : une machine peut essayer les 26 possibilités en microsecondes.

Problème 2.6 – *Jeu du plus/moins* ↑ énoncé

Algorithme 62 – Jeu du plus/moins

$n \leftarrow$ entier aléatoire dans $[1, 100]$; $nb_essais \leftarrow 0$;
Répéter
  Lire*prop* ; $nb_essais \leftarrow nb_essais + 1$;
  Si $prop < n$ **alors**
    Afficher« Plus »;
  Sinon si $prop > n$ **alors**
    Afficher« Moins »;
  Sinon
    Afficher« Trouvé en », nb_essais, « essais »;
  Fin si
Jusqu'à $prop = n$

La stratégie optimale est la **dichotomie** : toujours proposer le milieu de l'intervalle possible. Avec 100 nombres, on trouve en ≤ 7 essais car $\lceil \log_2 100 \rceil = 7$.

Problème 2.7 – *Crible d'Ératosthène* ↑ énoncé

Algorithme 63 – Crible d'Ératosthène

Entrée : entier $N \geq 2$
Créer un tableau $est_premier[2..N]$ initialisé à **Vrai**;
$p \leftarrow 2$;
Tant que $p^2 \leq N$ **faire**
  Si $est_premier[p]$ **alors**
    $k \leftarrow p^2$;
    Tant que $k \leq N$ **faire**
      $est_premier[k] \leftarrow$ **Faux**;
      $k \leftarrow k + p$;
    Fin tant que
  Fin si
  $p \leftarrow p + 1$;
Fin tant que
Afficher tous les i tels que $est_premier[i]$;

1. Pseudo-code : voir ci-dessus.
2. Justification de p^2 : si p est premier et $k < p^2$, alors k a déjà été barré par un premier $q < p$.
3. Le crible évite de tester séparément tous les diviseurs possibles de chaque entier.
4. Nombre de premiers ≤ 100 : 25.

Problème 2.8 – *Tri fusion : principe* ↑ énoncé

Fusion de deux tableaux triés :

Algorithme 64 – Fusion

Entrée : Tableaux triés $A[1..p]$ et $B[1..q]$
$i \leftarrow 1$; $j \leftarrow 1$; $k \leftarrow 1$;
Tant que $i \leq p$ **et** $j \leq q$ **faire**
  Si $A[i] \leq B[j]$ **alors**
    $C[k] \leftarrow A[i]$; $i \leftarrow i + 1$;
  Sinon
    $C[k] \leftarrow B[j]$; $j \leftarrow j + 1$;
  Fin si
  $k \leftarrow k + 1$;
Fin tant que
Copier les éléments restants de A ou B dans C;

Schéma récursif :

Algorithme 65 – Tri fusion (schéma)

Entrée : Tableau $T[g..d]$
Si $g < d$ **alors**
  $m \leftarrow \lfloor (g + d)/2 \rfloor$;
  Trier récursivement $T[g..m]$ et $T[m + 1..d]$;
  Fusionner $T[g..m]$ et $T[m + 1..d]$;
Fin si

Le tri fusion partage d'abord la liste, puis fusionne les sous-listes déjà triées.

Problème 2.9 – *Chiffrement de Vigenère* ↑ énoncé

Algorithme 66 – Chiffrement de Vigenère

Lire*cle* (longueur L), *message*;
Pour $i \leftarrow 1$ **à**longueur(*message*) **faire**
  $c \leftarrow$ rang du caractère i du message (0..25);
  $k \leftarrow$ rang du caractère $((i - 1) \bmod L + 1)$ de la clé;
  Afficherlettre de rang $(c + k) \bmod 26$;
Fin pour

Exemple : clé « CLE » (rangs 2,11,4). « BONJOUR » → « DZRJYAS » : $B(1) + C(2) = D(3)$; $O(14) + L(11) = Z(25)$; $N(13) + E(4) = R(17)$; etc.

Le chiffrement de Vigenère est plus sûr que César car il dépend d'une clé : 26^L clés possibles pour une clé de longueur L.

Problème 2.10 – *Compression RLE* ↑ énoncé

Algorithme 67 – Compression RLE

Entrée : Chaîne s
$i \leftarrow 1$;
Tant que $i \leq$ longueur(s) **faire**
  $c \leftarrow s[i]$; $count \leftarrow 1$;
  Tant que $i + count \leq$ longueur(s) **et** $s[i + count] = c$ **faire**
    $count \leftarrow count + 1$;
  Fin tant que
  Afficher*count*, c;
  $i \leftarrow i + count$;
Fin tant que

Exemple : `AAABBBBA` : `3A4B1A`.
La décompression lit un chiffre n suivi d'un caractère c et écrit c n fois. Le RLE est efficace pour les séquences avec beaucoup de répétitions (images, texte simple) mais peut *augmenter* la taille pour des séquences variées.

Problème 2.11 – *Correction logique d'un algorithme* ↑ énoncé

1. $L = \{3, 7, 2, 9, 5\}$: max = 9. L'algorithme donne 9 : correct ici.
2. $L = \{-4, -1, -7, -2\}$: toutes les valeurs sont négatives. L'algorithme initialise `max`$= 0$: aucune valeur n'est > 0, donc `max` reste 0. Mais le vrai maximum est -1. **Bug : initialisation incorrecte.**
3. La proposition est **fausse** : contre-exemple $L = \{-5, -3, -1\}$ donne 0 au lieu de -1.
4. Correction : initialiser `max`$\leftarrow L[1]$ (premier élément).
5. Invariant correct : « après i itérations, `max` est le maximum de $L[1..i]$ ».

Problème 2.12 – *Calcul de π par méthode de Monte-Carlo* ↑ énoncé

Algorithme 68 – Monte-Carlo pour π

Lire*N* ; $dans \leftarrow 0$;
Pour $i \leftarrow 1$ **à***N* **faire**
  $x \leftarrow$ réel aléatoire dans $[0, 1]$;
  $y \leftarrow$ réel aléatoire dans $[0, 1]$;
  Si $x^2 + y^2 \leq 1$ **alors**
    $dans \leftarrow dans + 1$;
  Fin si
Fin pour
Afficher$4 \times dans/N$;

N	Approximation typique	Erreur
100	3,1	±0,1
1 000	3,14	±0,03
100 000	3,141	±0,003

La convergence est lente, mais la méthode est facile à implémenter et peut être adaptée à d'autres simulations.

Problème 2.13 – *Statistiques par algorithme* ↑ *énoncé*

```python
def statistiques():
    n = int(input("Nombre de notes : "))
    notes = [float(input(f"Note {i+1} : ")) for i in range(n)]

    moyenne = sum(notes) / n
    mini = min(notes)
    maxi = max(notes)
    etendue = maxi - mini

    notes_triees = sorted(notes)
    if n % 2 == 1:
        mediane = notes_triees[n // 2]
    else:
        mediane = (notes_triees[n//2 - 1] + notes_triees[n//2]) / 2

    print(f"Moyenne : {moyenne:.2f}")
    print(f"Min : {mini}, Max : {maxi}, Etendue : {etendue}")
    print(f"Mediane : {mediane}")
```

Pour le tracé de l'histogramme, on peut utiliser `matplotlib` : `plt.hist(notes, bins=5)` produit un histogramme en 5 classes.

Défis

Défi 1 – *Tri par sélection*

Algorithme 69 – Tri par sélection

Entrée : Tableau $T[1..n]$
Pour $i \leftarrow 1$ **à** $n-1$ **faire**
  $imin \leftarrow i$;
  Pour $j \leftarrow i+1$ **à** n **faire**
    Si $T[j] < T[imin]$ **alors**
      $imin \leftarrow j$;
    Fin si
  Fin pour
  Échanger $T[i]$ et $T[imin]$;
Fin pour

Trace pour $[5,3,1,4,2]$: $i=1$: min en pos. 3 (val. 1), échanger : $[1,3,5,4,2]$. $i=2$: min en pos. 5 (val. 2) : $[1,2,5,4,3]$. $i=3$: min en pos. 5 (val. 3) : $[1,2,3,4,5]$. Terminé.

Nombre de comparaisons : $(n-1)+(n-2)+\cdots+1 = n(n-1)/2$. Le tri effectue au plus $n-1$ échanges, puisqu'un élément est placé définitivement à chaque passage.

Défi 2 – *Flocon de Koch*

Le flocon de Koch se construit en itérant sur chaque segment : le diviser en 3, retirer le tiers central et y placer un triangle équilatéral.

Algorithme 70 – Flocon de Koch (simulation)

nombre_segments $\leftarrow 3$; *longueur* $\leftarrow 1$;
Pour $k \leftarrow 1$ **à** N **faire**
  nombre_segments $\leftarrow 4 \times$ *nombre_segments*;
  longueur $\leftarrow$ *longueur*$/3$;
  perimetre $\leftarrow$ *nombre_segments* $\times$ *longueur*;
  Afficher« Itération : », k, « Périmètre : », *perimetre*;
Fin pour

Après k itérations : 3×4^k segments de longueur $(1/3)^k$. Périmètre : $3 \times (4/3)^k \to +\infty$ (le périmètre est infini !). Aire : $\frac{2\sqrt{3}}{5}$ (finie). C'est un exemple de **fractale** : courbe de longueur infinie enfermant une aire finie.

Défi 3 – *Problème des 4 reines*

Algorithme 71 – 4 reines par force brute

Pour $c_1 \leftarrow 1$ **à**4 **faire**
  Pour $c_2 \leftarrow 1$ **à**4 **faire**
    Pour $c_3 \leftarrow 1$ **à**4 **faire**
      Pour $c_4 \leftarrow 1$ **à**4 **faire**
        Si configuration valide(c_1, c_2, c_3, c_4) **alors**
          Afficherc_1, c_2, c_3, c_4;
        Fin si
      Fin pour
    Fin pour
  Fin pour
Fin pour

La condition « configuration valide » vérifie : toutes les colonnes distinctes ; pas de diagonale commune ($|c_i - c_j| \neq |i - j|$ pour $i \neq j$).

Solutions pour 4×4 : $\{2, 4, 1, 3\}$ et $\{3, 1, 4, 2\}$ (exactement 2 solutions).

Pour 8×8, il y a 92 solutions : la force brute serait lente ; on utilise le retour sur trace (backtracking).

Défi 5 – *Automate distributeur*

Algorithme 72 – Automate distributeur

solde $\leftarrow 0$;
Tant que Vrai faire
  Afficher« Solde : », *solde*, « € »;
  Lire*action*;
  Si *action* = « inser » **alors**
    Lire*piece* ; *solde* $\leftarrow$ *solde* + *piece*;
  Sinon si *action* = « choisir » **alors**
    Lire*prix*;
    Si *solde* $\geq$ *prix* **alors**
      Afficher« Distribuition »;
      solde $\leftarrow$ *solde* $-$ *prix*;
      Si *solde* > 0 **alors**